MCV Lab Standard: Scalable ML Experimentation

Reproducibility, Config Management, and HPC Execution (Argparse + Hydra + SLURM)

Medical Computer Vision Lab (MCV Lab)

October 16, 2026

1 Purpose and Scope

This document defines the MCV Lab standard for running machine learning experiments in a scalable, reproducible, and collaborative way. It applies to:

- all training scripts (vision, time-series, multimodal)
- all experiment pipelines (pretrain, finetune, evaluate, inference)
- all compute platforms (local GPU workstation, lab server, HPC/SLURM)

2 Core Principles

2.1 Non-negotiables

P1. No code edits for experiments.
Experiments must be configured through CLI arguments and/or config overrides only.

P2. Full reproducibility.
Every run must be repeatable from a stored configuration.

P3. Traceability.
Every run must save: configuration, launch command, outputs, checkpoints.

P4. Scalability.
The code must support sweeps, multiple datasets, and automation (bash/SLURM/Hydra multirun).

2.2 Why this matters

The lab frequently runs hundreds of experiments (hyperparameter sweeps, ablations, domain shifts, modality missingness). Without strict standards, results become:

- not reproducible
- hard to interpret
- difficult to compare fairly
- expensive in wasted compute time

3 Experiment Taxonomy: When to Use Argparse vs Hydra

3.1 Recommended default

- **Argparse** for small single-script projects and quick baselines.
- **Hydra** for any project expected to scale (recommended).

3.2 Decision table

Scenario	Recommended Tool
Quick baseline training / single dataset	Argparse
Small ablation study (10–20 runs)	Argparse or Hydra
Multiple datasets + models + optimizers	Hydra
Multimodal fusion experiments	Hydra
Continual learning setups	Hydra
Large sweeps on SLURM	Hydra + SLURM arrays

4 Standard Run Output Structure

All experiments must follow:

```
project/
  runs/
    <run_name>/
      config.json OR config_resolved.yaml
      cmd.txt
      checkpoints/
        last.ckpt
        best.ckpt
      logs/
        metrics.jsonl OR tensorboard OR wandb
```

4.1 Minimum required artifacts

Every run directory must include:

- `cmd.txt`: exact command used to launch the experiment
- resolved config (`config.json` or `config_resolved.yaml`)
- at least one checkpoint

5 Lab Rule: No Code Changes for Experiments

5.1 Allowed

- `python train.py --lr 1e-4 --seed 2`
- `python train.py --config configs/baseline.json --batch_size 64`
- `python train.py model=resnet50 optim.lr=5e-5` (Hydra)

5.2 Disallowed

- editing learning rate/batch size/epochs in `train.py`
- hardcoding dataset paths
- commenting/uncommenting blocks to switch models

6 Reproducibility Standard

6.1 Required

All training scripts must support:

- `seed` argument
- determinism toggle (`--deterministic` or Hydra field)

- saving configuration and command

6.2 Recommended

- log git commit hash into run directory
- log environment: Python version, torch version, CUDA version

7 Part I: Argparse Standard (Baseline)

7.1 Why argument parsers scale experimentation

Argument parsers enable scalability because they:

- remove the need for code edits
- enable easy sweeps via bash/SLURM
- enforce safe typed arguments (avoid invalid runs)
- support config-file driven defaults with CLI overrides

7.2 Required arguments (minimum)

Every argparse training script should include:

- Experiment: `--run_name`, `--out_dir`
- Dataset: `--dataset`, `--data_dir`, `--num_workers`
- Model: `--model`, `--pretrained`
- Optimization: `--epochs`, `--batch_size`, `--lr`, `--weight_decay`
- Reproducibility: `--seed`, `--deterministic`
- Hardware: `--device`, `--amp`
- Resume: `--resume`
- Config support: `--config` (mandatory)

7.3 Reference template (argparse)

Below is the lab reference entrypoint template (students may adapt but preserve principles):

```
#!/usr/bin/env python3
"""
train.py - scalable training entrypoint template
"""

from __future__ import annotations

import argparse
import json
import random
import shutil
import sys
from dataclasses import asdict, dataclass
from datetime import datetime
from pathlib import Path
from typing import Any

import numpy as np

try:
    import torch
except ImportError:
    torch = None
```

```
def set_seed(seed: int, deterministic: bool = False) -> None:
    random.seed(seed)
    np.random.seed(seed)
    if torch is not None:
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        if deterministic:
            torch.backends.cudnn.deterministic = True
            torch.backends.cudnn.benchmark = False


@dataclass
class TrainConfig:
    run_name: str
    out_dir: str
    dataset: str
    data_dir: str
    model: str
    epochs: int
    batch_size: int
    lr: float
    weight_decay: float
    seed: int
    deterministic: bool
    device: str
    amp: bool
    resume: str | None


def timestamp() -> str:
    return datetime.now().strftime("%Y%m%d_%H%M%S")


def ensure_dir(path: str | Path) -> Path:
    path = Path(path)
    path.mkdir(parents=True, exist_ok=True)
    return path


def dump_json(path: Path, obj: dict[str, Any]) -> None:
    with open(path, "w", encoding="utf-8") as f:
        json.dump(obj, f, indent=2, sort_keys=True)


def load_json(path: str | Path) -> dict[str, Any]:
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(
        formatter_class=argparse.ArgumentDefaultsHelpFormatter
    )
    parser.add_argument("--run_name", type=str, default="")
    parser.add_argument("--out_dir", type=str, default="runs")
```

```
    parser.add_argument("--dataset", type=str, default="dummy",
                        choices=["dummy", "ptbxl", "mimic_ecg", "
                           rsna_trauma"])
    parser.add_argument("--data_dir", type=str, default="data")

    parser.add_argument("--model", type=str, default="resnet18",
                        choices=["resnet18", "resnet50", "vit_b16"])

    parser.add_argument("--epochs", type=int, default=30)
    parser.add_argument("--batch_size", type=int, default=16)
    parser.add_argument("--lr", type=float, default=1e-4)
    parser.add_argument("--weight_decay", type=float, default=1e-4)

    parser.add_argument("--seed", type=int, default=0)
    parser.add_argument("--deterministic", action="store_true")

    parser.add_argument("--device", type=str, default="cuda",
                        choices=["cuda", "cpu", "mps"])
    parser.add_argument("--amp", action="store_true")

    parser.add_argument("--resume", type=str, default=None)

    parser.add_argument("--config", type=str, default=None,
                        help="Optional JSON config to load defaults
                           from.")

    return parser


def parse_args_with_config(parser: argparse.ArgumentParser, argv: list[
    str]) -> argparse.Namespace:
    partial, _ = parser.parse_known_args(argv)
    if partial.config is not None:
        cfg = load_json(partial.config)
        parser.set_defaults(**cfg)
    return parser.parse_args(argv)


def main(argv: list[str]) -> int:
    parser = build_parser()
    args = parse_args_with_config(parser, argv)

    if args.run_name.strip() == "":
        args.run_name = f"{args.dataset}_{args.model}_lr{args.lr}_{
           timestamp()}"

    run_dir = ensure_dir(Path(args.out_dir) / args.run_name)
    ensure_dir(run_dir / "checkpoints")

    if args.config is not None:
        shutil.copy(args.config, run_dir / "config_used.json")

    cfg = TrainConfig(
        run_name=args.run_name,
        out_dir=str(run_dir),
        dataset=args.dataset,
        data_dir=args.data_dir,
        model=args.model,
```

```
        epochs=args.epochs,
        batch_size=args.batch_size,
        lr=args.lr,
        weight_decay=args.weight_decay,
        seed=args.seed,
        deterministic=args.deterministic,
        device=args.device,
        amp=args.amp,
        resume=args.resume,
    )

    dump_json(run_dir / "config.json", asdict(cfg))
    (run_dir / "cmd.txt").write_text("python " + " ".join(argv) + "\n")

    # TODO: training logic
    print(f"Training would run here. Outputs: {run_dir}")
    return 0


if __name__ == "__main__":
    raise SystemExit(main(sys.argv[1:]))
```

8 Part II: Hydra Standard (Recommended)

8.1 Why Hydra scales better than argparse

Hydra enables:

- hierarchical configs (dataset/model/optim/trainer)
- composable config groups
- built-in multi-run sweeps (`-m`)
- automatic run directory management and config saving

8.2 Required project structure

```
project/
  train.py
  configs/
    config.yaml
    dataset/
      rsna_trauma.yaml
      mimic_ecg.yaml
    model/
      resnet50.yaml
      vit_b16.yaml
    optim/
      adamw.yaml
      sgd.yaml
    trainer/
      default.yaml
```

8.3 Hydra entrypoint template

```
#!/usr/bin/env python3
from __future__ import annotations
```

```python
import os
import random
from pathlib import Path

import numpy as np
import hydra
from omegaconf import DictConfig, OmegaConf

try:
    import torch
except ImportError:
    torch = None


def set_seed(seed: int, deterministic: bool = False) -> None:
    random.seed(seed)
    np.random.seed(seed)
    if torch is not None:
        torch.manual_seed(seed)
        torch.cuda.manual_seed_all(seed)
        if deterministic:
            torch.backends.cudnn.deterministic = True
            torch.backends.cudnn.benchmark = False


@hydra.main(version_base=None, config_path="configs", config_name="
    config")
def main(cfg: DictConfig) -> None:
    run_dir = Path(os.getcwd())

    (run_dir / "config_resolved.yaml").write_text(
        OmegaConf.to_yaml(cfg), encoding="utf-8"
    )

    set_seed(int(cfg.seed), deterministic=bool(cfg.deterministic))

    print("Run dir:", run_dir)
    print(OmegaConf.to_yaml(cfg))

    # TODO: build dataset/model/optim/trainer based on cfg groups


if __name__ == "__main__":
    main()
```

8.4 Hydra base config

```yaml
# configs/config.yaml
defaults:
  - dataset: rsna_trauma
  - model: resnet50
  - optim: adamw
  - trainer: default
  - _self_

seed: 0
deterministic: false
```

8.5 Example Hydra multirun sweeps

```
python train.py -m optim.lr=1e-3,1e-4,1e-5 seed=0,1,2
python train.py -m model=resnet50,vit_b16 dataset=rsna_trauma,mimic_ecg
```

9 Part III: SLURM Standard (HPC Execution)

9.1 Required folder structure

```
project/
  runs/
  slurm_logs/
  scripts/
    run_train.slurm
    sweep_seeds.slurm
    ddp_train.slurm
```

9.2 Baseline SLURM job (single run)

```
#!/bin/bash
#SBATCH --job-name=mcv_train
#SBATCH --output=slurm_logs/%x_%j.out
#SBATCH --error=slurm_logs/%x_%j.err
#SBATCH --time=08:00:00
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G

set -euo pipefail

echo "JOB_ID: ${SLURM_JOB_ID}"
echo "NODELIST: ${SLURM_NODELIST}"
echo "START_TIME: $(date)"

cd ${SLURM_SUBMIT_DIR}
nvidia-smi || true

python train.py \
  --dataset rsna_trauma \
  --model resnet50 \
  --lr 1e-4 \
  --seed 0 \
  --amp \
  --out_dir runs \
  --run_name "rsna_resnet50_seed0_lr1e4_${SLURM_JOB_ID}"

echo "END_TIME: $(date)"
```

9.3 SLURM arrays for sweeps (recommended)

```
#!/bin/bash
#SBATCH --job-name=mcv_seed_sweep
#SBATCH --output=slurm_logs/%x_%A_%a.out
```

```
#SBATCH --error=slurm_logs/%x_%A_%a.err
#SBATCH --time=08:00:00
#SBATCH --partition=gpu
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=8
#SBATCH --mem=32G
#SBATCH --array=0-4

set -euo pipefail
cd ${SLURM_SUBMIT_DIR}

SEEDS=(0 1 2 3 4)
SEED=${SEEDS[$SLURM_ARRAY_TASK_ID]}

python train.py \
  --dataset rsna_trauma \
  --model resnet50 \
  --lr 1e-4 \
  --seed ${SEED} \
  --amp \
  --out_dir runs \
  --run_name "rsna_resnet50_seed${SEED}_job${SLURM_JOB_ID}"
```

9.4 Hydra + SLURM (best practice)

Use SLURM arrays for large sweeps, Hydra overrides for config control:

```
python train.py dataset=rsna_trauma model=resnet50 optim=adamw optim.lr
   =1e-4 seed=${SEED}
```

9.5 Multi-GPU training (DDP)

```
#!/bin/bash
#SBATCH --job-name=mcv_ddp
#SBATCH --output=slurm_logs/%x_%j.out
#SBATCH --error=slurm_logs/%x_%j.err
#SBATCH --time=12:00:00
#SBATCH --partition=gpu
#SBATCH --gres=gpu:2
#SBATCH --cpus-per-task=12
#SBATCH --mem=64G

set -euo pipefail
cd ${SLURM_SUBMIT_DIR}

export OMP_NUM_THREADS=${SLURM_CPUS_PER_TASK}

torchrun --nproc_per_node=2 train.py \
  --dataset rsna_trauma \
  --model resnet50 \
  --lr 1e-4 \
  --seed 0 \
  --amp \
  --out_dir runs \
  --run_name "rsna_ddp2_${SLURM_JOB_ID}"
```

10 Checklists

10.1 Checklist: experiment readiness

- Runs end-to-end without code edits
- Outputs go to `runs/<run_name>/`
- Saves resolved config + cmd
- Logs seed, dataset, model, learning rate
- Saves checkpoints (best + last)

10.2 Checklist: code review / PR checklist

Before merging experiment code:

- Argument/config surface is documented
- Dataset paths not hardcoded
- All new arguments have defaults and help messages
- Config groups exist (Hydra) for dataset/model/optim
- SLURM scripts write logs into `slurm_logs/`

11 Recommended Extensions (Optional)

- Use W&B or MLflow and log full config per run
- Save git commit hash in run directory
- Save environment info in `env.txt`
- Add automated experiment summary script (table of metrics across runs)